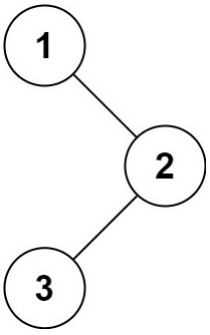
Binary Tree

leetcode

94.二叉树的中序遍历

- 遍历二叉树，返回中序遍历序列。



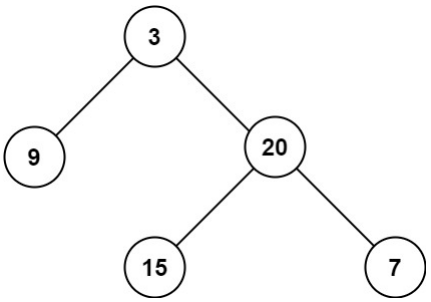
- 思路：递归遍历。左根右遍历序列。

```
class Solution {
public:
    void inorder(TreeNode* root,vector<int>& res){
        // 中序遍历-左根右
        if(!root){ // 终止条件
            return;
        }
        // 左结点
        inorder(root->left,res);
        // 根节点
        res.push_back(root->val);
        // 右结点
        inorder(root->right,res);
    }

    vector<int> inorderTraversal(TreeNode* root) {
        vector<int> res;
        inorder(root,res);
        return res;
    }
};
```

-

95.二叉树的最大深度

- 给定一个二叉树 `root` ，返回其最大深度（根节点到最远叶节点之间的节点数）。




```
输入: root = [3,9,20,null,null,15,7]
输出: 3
```

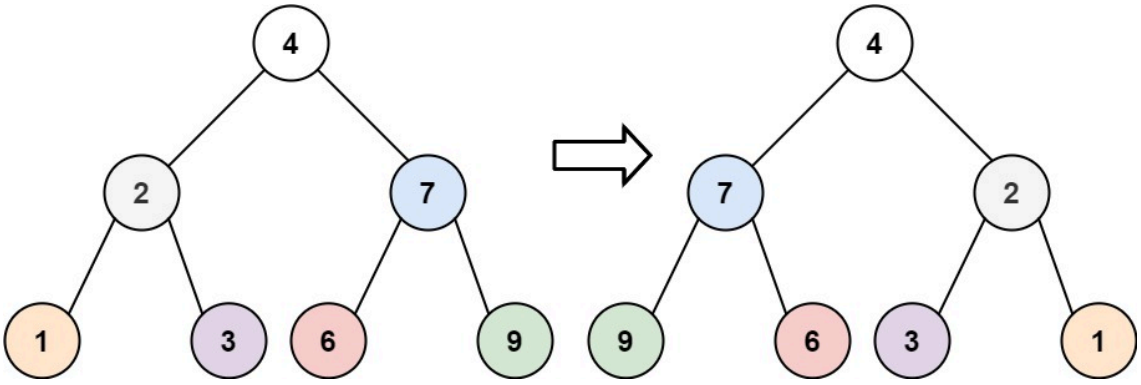
- 思路：递归遍历。最大深度=1+max(左子树最大深度，右子树最大深度)。

```cpp
class Solution {
public:
    int maxDepth(TreeNode* root) {
        if(root == nullptr) {
            return 0;
        } else {
            int left = maxDepth(root->left);
            int right = maxDepth(root->right);
            return std::max(left, right) + 1;
        }
    }
};
```

-

226.翻转二叉树

- 反转二叉树，镜像对称。




```
输入: root = [4,2,7,1,3,6,9]
输出: [4,7,2,9,6,3,1]
```

- 思路：递归遍历。先将左右子树均反转完，然后再交换当前节点的左右子树。

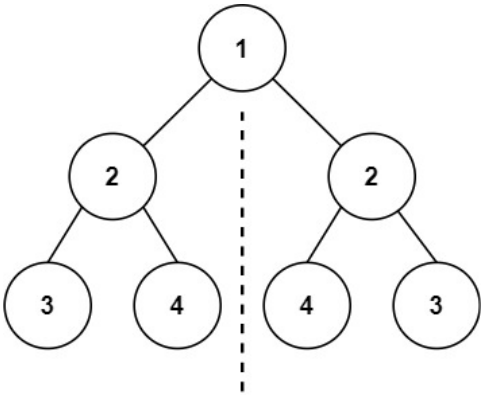
```cpp
class Solution {
public:
    TreeNode* invertTree(TreeNode* root) {
        if( root==nullptr ) return nullptr; // 终止条件
        // 反转左子树
        TreeNode *left = invertTree(root->left);
        // 反转右子树
        TreeNode *right = invertTree(root->right);
```

```
        // 交换当前节点的左右子树
        root->left = right;
        root->right = left;
        return root;
    }
};
```

-

101.对称二叉树

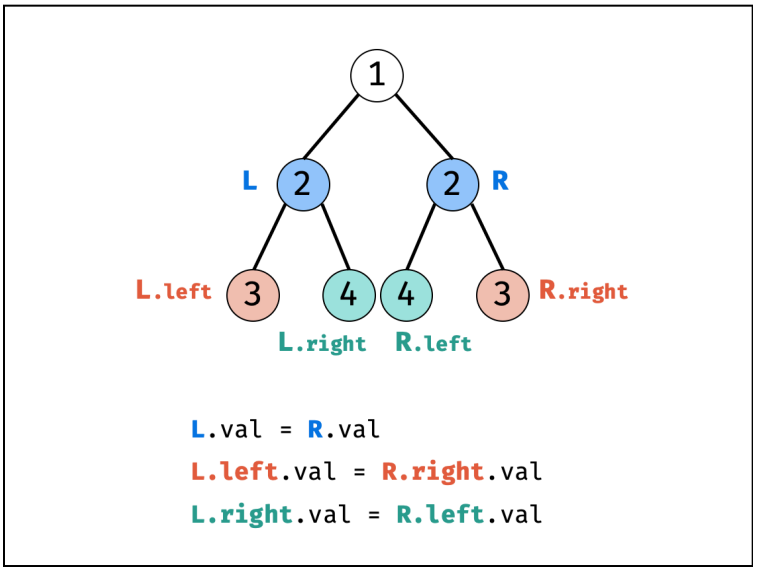
- 检查一棵二叉树是否轴对称。



- 思路：递归遍历。

函数 `recur(L, R)` **：**

- **终止条件：**
  - 当 `L` 和 `R` 同时越过叶节点： 此树从顶至底的节点都对称，因此返回 *true* 。
  - 当 `L` 或 `R` 中只有一个越过叶节点： 此树不对称，因此返回 *false* 。
  - 当节点 `L` 值 $\neq$ 节点 `R` 值： 此树不对称，因此返回 *false* 。
- **递推工作：**
  - 判断两节点 `L.left` 和 `R.right` 是否对称，即 `recur(L.left, R.right)` 。
  - 判断两节点 `L.right` 和 `R.left` 是否对称，即 `recur(L.right, R.left)` 。
- **返回值：** 两对节点都对称时，才是对称树，因此用与逻辑符 `&&` 连接。

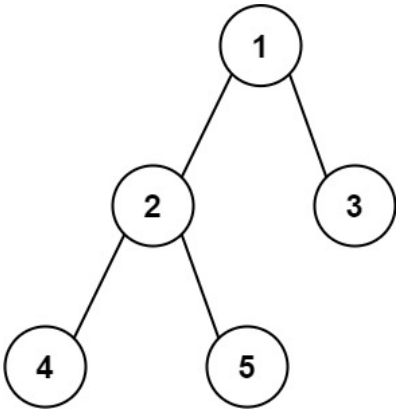


```
class Solution {
public:
    bool isSymmetric(TreeNode* root) {
        if(root==nullptr) return true;
        else return recur(root->left,root->right);
    }
    bool recur(TreeNode* L,TreeNode* R){
        if(L==nullptr&&R==nullptr) return true;
        if(L==nullptr||R==nullptr||L->val!=R->val) return false;
        return recur(L->left,R->right)&&recur(R->left,L->right);
    }
};
```

-

543.二叉树的直径

- 给一棵二叉树，返回树的直径。二叉树的 **直径** 是指树中任意两个节点之间最长路径的 **长度** 。



输入: root = [1,2,3,4,5]
输出: 3
解释: 3 , 取路径 [4,2,1,3] 或 [5,2,1,3] 的长度。

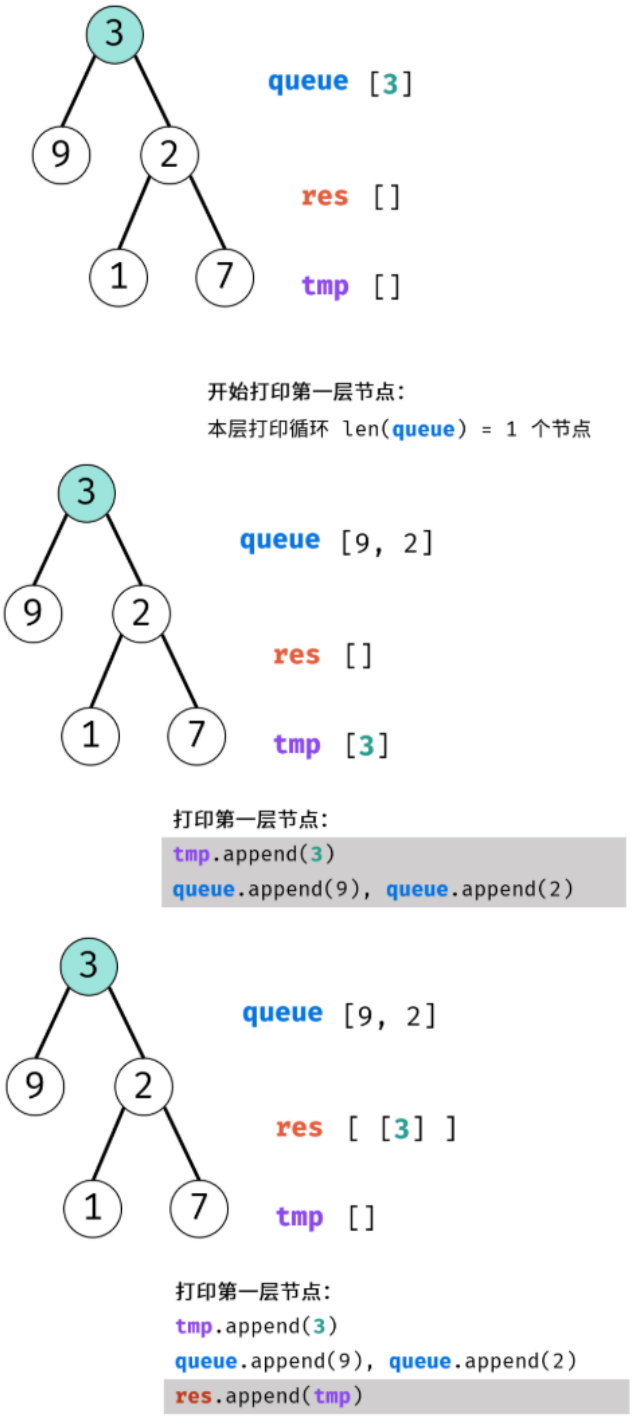
- 思路：左子树最大长度+右子树最大长度

```
class Solution {
    int res = 0; // 存储全局最大值
public:
    int diameterOfBinaryTree(TreeNode* root) {
        res = 0;
        maxDepth(root);
        return res;
    }

    int maxDepth(TreeNode * root){
        if(root==nullptr) return 0;
        int LH = maxDepth(root->left);
        int RH = maxDepth(root->right);
        res = max(res,LH+RH); // 局部最大值，经过当前 root 节点的路径长度=左深度 +右深度
        return 1+std::max(LH,RH); // 最大深度
    }
};
```

-

102.二叉树的层序遍历

- 返回二叉树层序遍历的结果。
- 思路：二叉树的BFS广度优先搜索。借助**队列**来实现。将本层全部节点打印到一行，并将下一层全部节点加入队列，以此类推，即可分为多行打印。



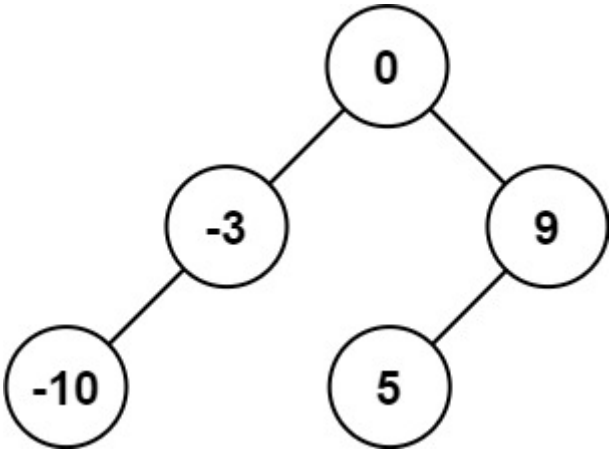

```
class Solution {
public:
    vector<vector<int>> levelOrder(TreeNode* root) {
        vector<vector<int>> res;
        queue<TreeNode*> q;
        if (root != nullptr) q.push(root);
        while(!q.empty()){
```

```
            vector<int> temp; // 用来存储该层的结果
            int levelSize = q.size(); // 当前层的元素个数
            for(int i=0;i<levelSize;i++){
                TreeNode* head = q.front();
                q.pop();
                temp.push_back(head->val);
                // 将该层结点的左右孩子压入队列中
                if( head->left!=nullptr ) q.push(head->left);
                if( head->right!=nullptr ) q.push(head->right);
            }
            res.push_back(temp);
        }
        return res;
    }
};
```

-

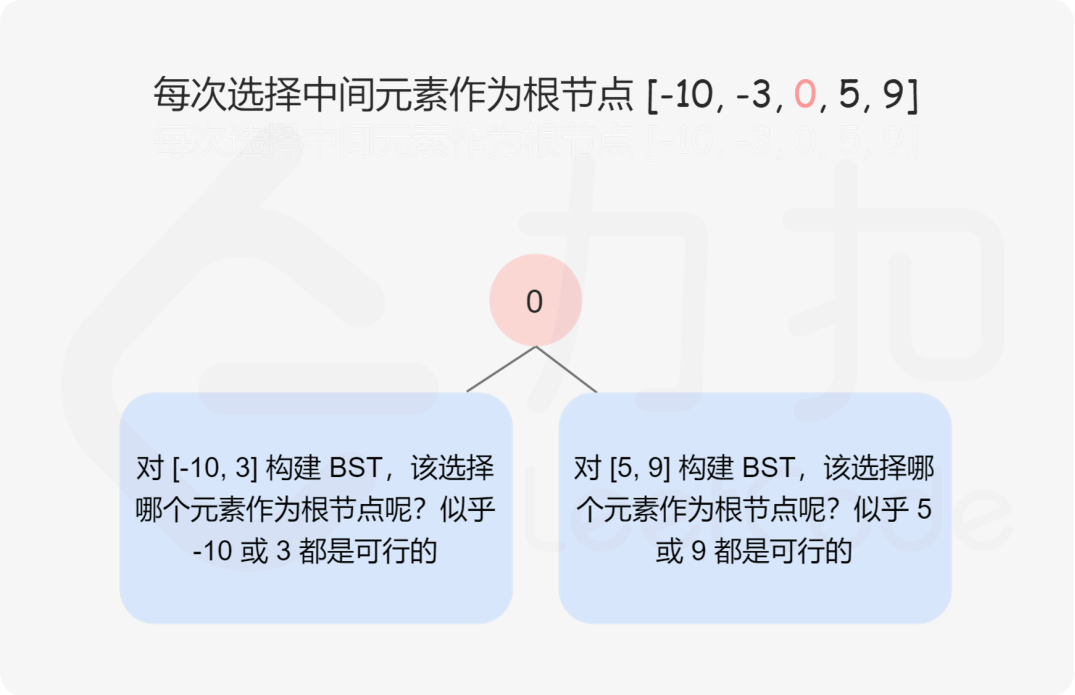
108.将有序数组转换为二叉搜索树

- 给定一个按升序排列的数组，转换为平衡二叉树（**平衡二叉树** 是指该树所有节点的左右子树的高度相差不超过 1，左子节点 < 根节点 < 右子节点，二叉搜索树的中序遍历是升序序列）。




```
输入: nums = [-10,-3,0,5,9]
输出: [0,-3,9,-10,null,5]
```

- 思路：
  - 首先确定**二叉搜索树的根节点**。选择**中间数字**作为二叉搜索树的根节点，这样分给左右子树的数字个数相同或只相差 1，可以使得树保持平衡。



  - 如果数组长度是**奇数**，则根节点的选择是**唯一**的。
  - 如果数组长度是**偶数**，则可以选择**中间位置左边的数字**作为根节点或者选择**中间位置右边的数字**作为根节点。
- 左子树和右子树分别是平衡二叉搜索树，因此可以通过**递归的方式**创建平衡二叉搜索树。
- 三种方法
  - 中序遍历，总是选择中间位置左边的数字作为根节点——*mid*=(*left*+*right*)/2，整数除法。
  - 中序遍历，总是选择中间位置右边的数字作为根节点——*mid*=(*left*+*right*+1)/2
  - 中序遍历，选择任意一个中间位置数字作为根节点——*mid*=(*left*+*right*)/2 和 *mid*=(*left*+*right*+1)/2 两者中随机选择一个，mid = (left + right + rand.nextInt(2)) / 2;

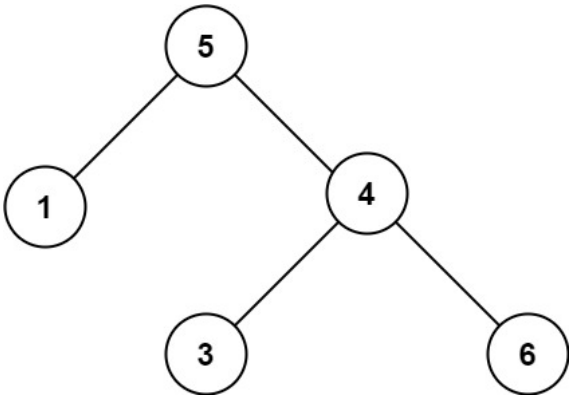
```
/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left),
right(right) {}
 * };
 */
class Solution {
public:
    TreeNode* sortedArrayToBST(vector<int>& nums) {
        return buildBST(nums,0,nums.size() - 1);
    }
    TreeNode* buildBST(vector<int>& nums,int left,int right){
        if( left>right ){
            return nullptr; // 终止条件
        }
        // 总是选择中间位置左边的数字作为根结点
        int mid = (left+right)/2; // 整数除法
        TreeNode* root = new TreeNode(nums[mid]);
```

```
        root->left = buildBST(nums,left,mid-1);// 左子树
        root->right = buildBST(nums,mid+1,right);// 右子树
        return root;
    }
};
```

-
-

98.验证二叉搜索树

- 二叉树的根节点 `root` ，判断其是否是一个有效的二叉搜索树。




```
输入: root = [5,1,4,null,null,3,6]
输出: false
解释: 根节点的值是 5 ，但是右子节点的值是 4 。
```

  - 节点的左子树只包含 **严格小于** 当前节点的数。
  - 节点的右子树只包含 **严格大于** 当前节点的数。
  - 所有左子树和右子树自身必须也是二叉搜索树。
- 思路：递归判断。
  - 对于根节点：范围是 $(-\infty, +\infty)$ 。
  - 对于左孩子：它的上界变成了父节点的值（因为它必须小于父节点），下界继承父节点的下界。
    - 新区间： `(lower, parent->val)`
  - 对于右孩子：它的下界变成了父节点的值（因为它必须大于父节点），上界继承父节点的上界。
    - 新区间： `(parent->val, upper)`

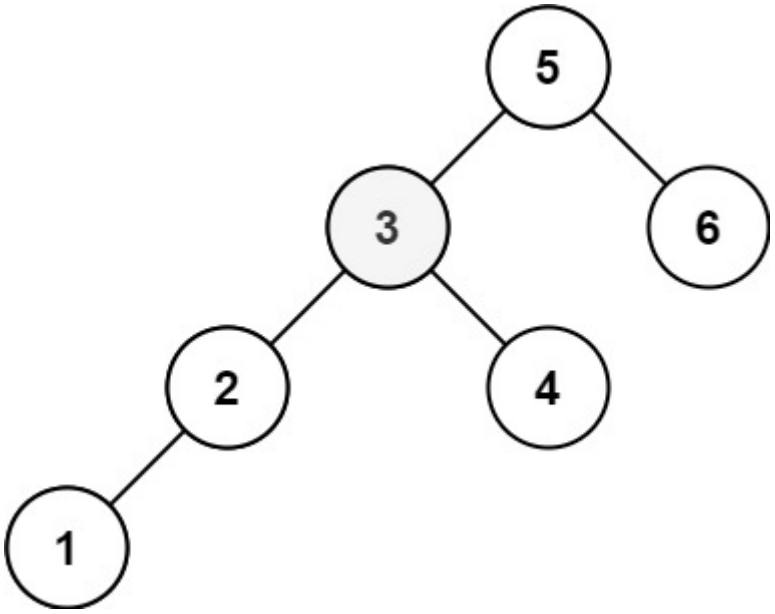
```
/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left),
right(right) {}
```

```
 * };
 */
class Solution {
public:
    bool isValidBST(TreeNode* root) {
        return check(root,LONG_MIN,LONG_MAX); // 根节点的范围在(-∞,+∞)
    }

    bool check(TreeNode* root, long long lower, long long upper){
        if(root==nullptr){
            return true;
        }
         // 当前节点的值必须严格大于下界，且严格小于上界
        if( root->val <= lower || root->val >= upper ){
            return false;
        }
        // 检查左右子树的合法性
        return check(root->left,lower,root->val)&&check(root->right,root-
>val,upper);
    }
};
```

-

230.二叉搜索树中第k小的元素

- 一个二叉搜索树的根节点 `root` ，和一个整数 `k` ，找到其中第k小的元素。




```
输入: root = [5,3,6,2,4,null,null,1], k = 3
输出: 3
```

- 思路：中序遍历，二叉搜索树 (BST) 的中序遍历 (In-order Traversal) 是按键值增加的有序序列。
    - BST 的中序遍历顺序： `左子树 -> 根节点 -> 右子树` 。
    - 遍历结果是一个从小到大排序的数组。
    - 因此，第 k 个被访问到的节点，就是第 k 小的元素。

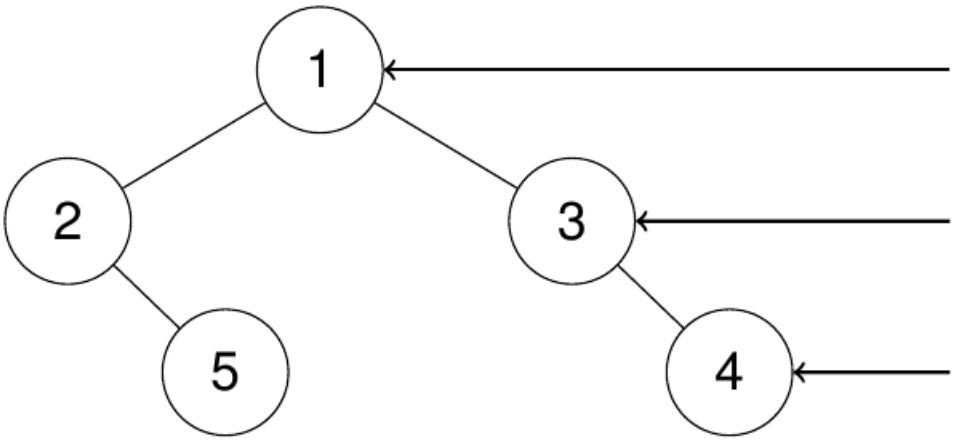
```
class Solution {
```

```
    int cnt = 0;
    int ans;
public:
    int kthSmallest(TreeNode* root, int k) {
        traverse(root,k);
        return ans;
    }

    void traverse(TreeNode* root, int k){
        if(root==nullptr) return;
        // 中序遍历-左根右
        traverse(root->left,k);
        cnt++;
        if(cnt==k){
            ans = root->val;
            return;
        }
        traverse(root->right,k);
    }
};
```

-

199.二叉树的右视图

- 一个二叉树的 **根节点** `root`，站在它的右侧，按照从顶部到底部的顺序，返回从右侧所能看到的节点值。




```
输入: root = [1,2,3,null,5,null,4]

输出: [1,3,4]
```

- 思路：层序遍历同时记录最后出队的元素。每一层的最右侧元素就是从右边看的结果。

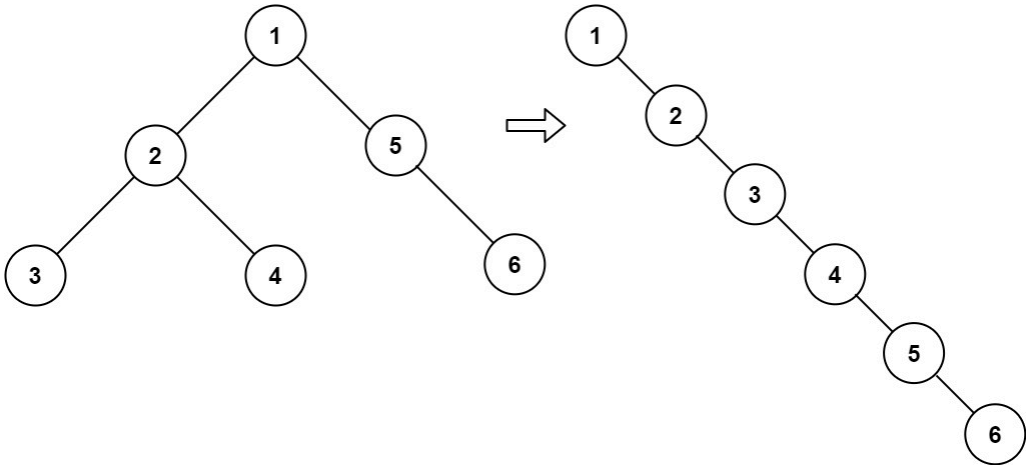
```
class Solution {
public:
    vector<int> rightSideView(TreeNode* root) {
        vector<int> result;
        if (root == nullptr) return result;
        queue<TreeNode*> Q;
```

```
        Q.push(root);
        while(!Q.empty()){
            int len = Q.size(); // 当前层的大小
            // 层序遍历，当遍历到最后一个结点时(len-1)，把结果存储res
            for(int i=0;i<len;i++){
                TreeNode * temp = Q.front();
                Q.pop();
                if(temp->left!=nullptr) Q.push(temp->left);
                if(temp->right!=nullptr) Q.push(temp->right);
                if(i == len - 1){
                    result.push_back(temp->val);
                }
            }
        }
        return result;
    }
};
```

-

114.二叉树展开为链表

- 将二叉树的根结点 `root` 展开为一个单链表。展开后的单链表应该与二叉树**先序遍历**顺序相同。单链表应该同样使用 `TreeNode` 。




```
输入: root = [1,2,5,3,4,null,6]
输出: [1,null,2,null,3,null,4,null,5,null,6]
```

- 思路： 将二叉树展开为单链表之后，单链表中的节点顺序即为二叉树的前序遍历访问各节点的顺序。

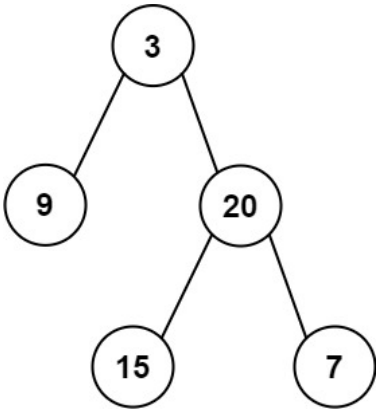
```
class Solution {
public:
    void flatten(TreeNode* root) {
        vector<TreeNode*> l;
        preorderTraversal(root, l); // 先序遍历序列就是展平后的结果
        // 遍历链表，把左结点改为null，右结点指向先序序列的下一结点
        for( int i=1;i<l.size();i++ ){
            TreeNode *prev = l[i-1];
            TreeNode *curr = l[i];
            prev->left=nullptr;
            prev->right = curr;
        }
```

```
    }

    void preorderTraversal(TreeNode* root, vector<TreeNode*> &l){
        if( root==NULL ){
            return;
        }
        l.push_back(root);
        preorderTraversal(root->left,l);
        preorderTraversal(root->right,l);
    }
};
```

-

105.从前序与中序遍历构造二叉树

- 两个整数数组 `preorder`(**先序遍历**) 和 `inorder` (**中序遍历**)。




```
输入: preorder = [3,9,20,15,7], inorder = [9,3,15,20,7]
输出: [3,9,20,null,null,15,7]
```

- 思路:

  1. 前序遍历 (Preorder): `[根, 左子树..., 右子树...]`
     - 作用: 第一个元素永远是当前的根节点。这让我们知道了"头"是谁。
  2. 中序遍历 (Inorder): `[左子树..., 根, 右子树...]`
     - 作用: 一旦我们知道了"根"是谁 (从前序拿到) , 就可以在中序数组中找到这个根的位置。根的左边全是左子树, 根的右边全是右子树。这让我们知道了"左右边界"在哪里。

  逻辑链条:

  1. 看前序数组的第一个数 -> 确定 根节点。
  2. 去中序数组找这个根节点 -> 确定 左子树有多少个节点 (`size_left`)。
  3. 根据 `size_left` , 在前序数组中切分出 左子树的前序区间 和 右子树的前序区间。
  4. 在中序数组中切分出 左子树的中序区间 和 右子树的中序区间。
  5. 递归重复上述步骤。

算法流程：

1. 定位根：取前序序列的第一个数 `root_val` 。
2. 分割中序：在中序序列中找到 `root_val` 的索引 `idx` 。
   - 中序左区间： `[in_left, idx-1]`
   - 中序右区间： `[idx+1, in_right]`
3. 计算大小：左子树的节点数量 `size = idx - in_left` 。
4. 分割前序：根据 `size` 切割前序序列。
   - 前序左区间： `[pre_left+1, pre_left+size]` （跳过根节点，取接下来的 size 个）
   - 前序右区间： `[pre_left+size+1, pre_right]` （剩下的都是右子树）
5. 递归构建：对左右子区间重复上述步骤，直到区间为空。

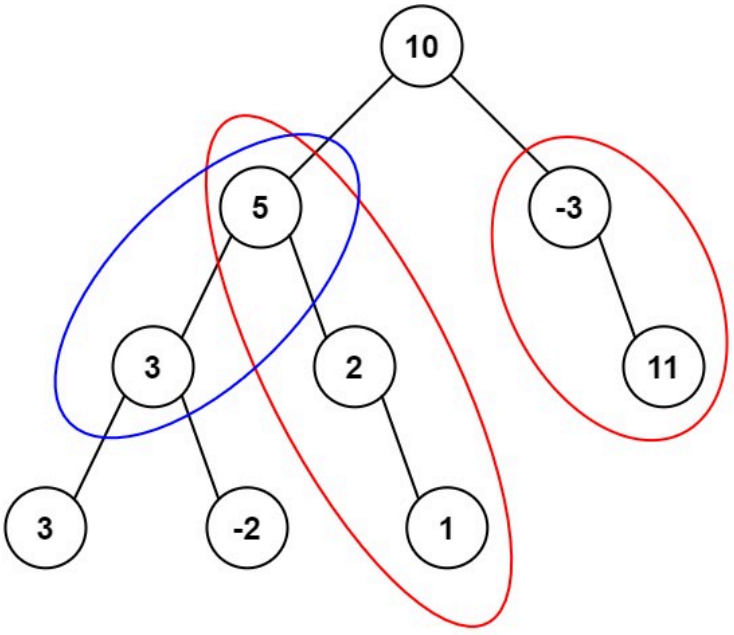
```
class Solution {
    unordered_map<int, int> nodeToIndex;
public:
    TreeNode* buildBT(vector<int>& preorder,vector<int>& inorder,
    int preorder_left, int preorder_right,
    int inorder_left, int inorder_right){
        if (preorder_left > preorder_right) {
            return nullptr;
        }

        int preorder_root = preorder_left; // 前序遍历中的第一个节点就是根节点
        int inorder_root = nodeToIndex[preorder[preorder_root]]; // 根据前序遍历结点找根节点在中序遍历中的位置
        // 根节点构造
        TreeNode* root = new TreeNode(preorder[preorder_root]);
        int left_subtree_size = inorder_root - inorder_left; // 从中序序列推出左子树的大小
        // 递归构造左子树
        // 先序遍历中「从 左边界+1 开始的 size_left_subtree」个元素——中序遍历中「从 左边界 开始到 根节点定位-1」的元素
        root->left = buildBT(preorder, inorder,
        preorder_left + 1, preorder_left + left_subtree_size,
        inorder_left, inorder_root - 1
        );
        // 递归构造右子树
        // 先序遍历中「从 左边界+1+左子树节点数目 开始到 右边界」的元素——中序遍历中「从 根节点定位+1 到 右边界」的元素
        root->right = buildBT(preorder, inorder,
        preorder_left + left_subtree_size + 1, preorder_right,
        inorder_root + 1, inorder_right
        );
        return root;

    }

    TreeNode* buildTree(vector<int>& preorder, vector<int>& inorder) {
        int n = preorder.size();
```

```
        // 构建哈希映射，用于快速定位中序遍历的根节点
        for( int i=0; i<n; i++ ){
            nodeToIndex[inorder[i]] = i;
        }
        return buildBT( preorder,inorder,
        0,n - 1,
        0, n - 1 );
    }
};
```

-

437.路径综合Ⅲ

- 一个二叉树的根节点 `root` ，和一个整数 `targetSum` ，求该二叉树里节点值之和等于 `targetSum` 的 **路径** 的数目。**路径** 不需要从根节点开始，也不需要在叶子节点结束，但是路径方向必须是向下的。




```
输入: root = [10,5,-3,3,2,null,11,3,-2,null,1], targetSum = 8
输出: 3
解释: 和等于 8 的路径有 3 条，如图所示。
```

- **思路**：**双重递归 (Double Recursion)。遍历树上的每一个节点**，把每个节点都当作"起点"，然后向下搜索有多少条路径的和等于 `targetSum`。
  - `dfs(node, current_sum)`：以 `node` 为**起点**，向下寻找和为 `targetSum` 的路径数量。
  - `pathSum(root, targetSum)`：遍历整棵树，对**每个节点**都调用一次 `dfs`。

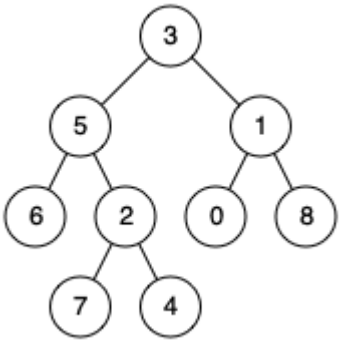
```
class Solution {
public:
    int dfs(TreeNode* root, long long currentSum,long long targetSum){
        if( !root ) return 0;
        int count = 0;
        // 根
        currentSum += root->val; // 更新当前路径长度
        if(currentSum == targetSum) count++;
```

```
        // 左
        count += dfs(root->left,currentSum,targetSum);
        // 右
        count += dfs(root->right,currentSum,targetSum);

        return count;
    }

    int pathSum(TreeNode* root, int targetSum) {
        // 遍历每个结点，将其作为起点，进行深度遍历，计算和为targetSum的路径数量
        if(!root) return 0;
        // 以当前节点为起点的和为targetSum的路径数
        int count = dfs(root,0,targetSum);
        // 递归计算左子树中所有结点作为起点的路径数
        count += pathSum(root->left,targetSum);
        // 递归计算右子树中所有结点作为起点的路径数
        count += pathSum(root->right,targetSum);

        return count;
    }
};
```

-

236.二叉树的最近公共祖先

- 给定一个二叉树, 找到该树中两个指定节点的**最近公共祖先**。




```
输入: root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 4
输出: 5
解释: 节点 5 和节点 4 的最近公共祖先是节点 5 。因为根据定义最近公共祖先节点可以为节点本身。
```

- 思路:

1. 什么是“最近公共祖先” (LCA)?

对于节点 `p` 和 `q`，LCA 是离它们最近的、同时拥有 `p` 和 `q` 作为后代的节点。
在二叉树中，LCA 只会出现以下三种情况：

1. 分叉型： `p` 在 `root` 的左子树， `q` 在 `root` 的右子树（或反之）。此时 `root` 就是 LCA。
2. 包含型 (p 是祖先)： `p` 就是 `root`，而 `q` 在 `p` 的子树中。此时 `p` (即 root) 就是 LCA。
3. 包含型 (q 是祖先)： `q` 就是 `root`，而 `p` 在 `q` 的子树中。此时 `q` (即 root) 就是 LCA。

```
```

-
-